A Configurable Dynamic-Fee Hook Framework for Uniswap v4 Liquidity Pools

***Abstract*—Automated market makers typically apply a static swap fee although volatility, order flow, and liquidity-provider exposure vary over time. Uniswap v4 makes fee logic programmable through hooks, but implementing and comparing alternative policies still requires protocol-specific contracts, deployment scripts, data feeds, simulations, and analysis tools. This paper presents a configurable framework that unifies these components. It provides a shared Solidity abstraction and templates for block-adaptive, deal-adaptive, asymmetric, MEV-related, and peg-stability fee policies; optional Chainlink-compatible price access; reusable Foundry/Anvil deployment and testing utilities; and a web configurator that exposes parameters and generated source code. The same interface exports a deployable project and runs historical-data replay using Binance observations. A prototype execution over an ETH/SHIB trace processes 1,492 simulated swaps and produces external-price, price-ratio, pool-price, volume, fee, and liquidity-provider metrics. The evidence establishes end-to-end implementation feasibility, not economic superiority over static fees. The framework contributes reproducible engineering infrastructure through which dynamic-fee policies can be inspected, deployed, and evaluated under a common workflow.**

***Index Terms*—automated market makers, decentralized exchanges, dynamic fees, Uniswap v4, smart-contract hooks, liquidity provision**

I. Introduction

Decentralized finance (DeFi) implements exchange, lending, derivatives, and asset-management services through publicly executable smart contracts [1]. Within this ecosystem, decentralized exchanges (DEXs) remove the need for a custodial order-matching intermediary. Many DEXs use constant-function automated market makers (AMMs), which accept trades against reserves according to a deterministic function [2], [3]. Uniswap popularized the constant-product design and subsequently introduced concentrated liquidity, allowing liquidity providers (LPs) to allocate capital within selected price ranges [4], [5]. These mechanisms enable permissionless market creation and continuous on-chain execution, but they also transfer inventory and adverse-selection risk to LPs. Although the term *dynamic pricing* is sometimes used broadly in this context, it is critical to clarify that this framework controls the trading fee parameter, not the asset price itself. In AMM systems, the price is determined by the pool invariant and liquidity state; this framework solely manages the adaptive rules for fee extraction to balance LP protection and trader execution costs.

An AMM price changes only when a transaction updates the pool state. When an external market moves first, arbitrageurs trade against the stale pool until the price discrepancy is no longer profitable. The resulting rebalancing changes the composition and value of the LP position. Impermanent loss compares that position with holding the deposited assets, whereas loss-versus-rebalancing (LVR) isolates the cost of being passively rebalanced at stale prices relative to a continuously rebalanced benchmark [6]. Concentrated liquidity adds a further allocation trade-off: narrow ranges increase fee share while active, but cease earning fees after the market exits the range and can require costly repositioning [7].

Swap fees are the primary compensation LPs receive for bearing these risks, directly impacting the magnitude of impermanent loss [8]. A static fee, however, must serve incompatible market states. During stable periods, a high fee worsens execution and can redirect order flow to competing venues. During volatile periods, a low fee may not compensate LPs for informed or arbitrage flow. The appropriate fee therefore depends on variables such as volatility, external-price displacement, pool depth, trade direction, and the amount of uninformed volume. Dynamic mechanisms attempt to price this changing exposure instead of selecting one fee ex ante.

Recent research formalizes this trade-off from several directions. Fritsch [9] analyzes optimal fee structures for constant function market makers, while Lebedeva *et al.* evaluate block-adaptive, deal-adaptive, and oracle-informed asymmetric fees [10]. Baggiani *et al.* derive dynamic policies through stochastic control [11], while Campbell *et al.* study optimal fees when an AMM competes with a centralized exchange [12]. Agent-based work further incorporates latency, heterogeneous traders, arbitrageurs, and maximal-extractable-value (MEV) searchers [13]. Collectively, these results motivate adaptive fees, but their economic models do not remove the engineering burden of implementing multiple policies consistently on a production-oriented AMM.

Uniswap v4 provides the missing protocol primitive. Its hooks are external contracts invoked at selected points of a pool's lifecycle; a dynamic-fee pool can update its stored LP fee or obtain a per-swap override from a hook [14], [15]. Yet a usable experiment requires more than the fee equation. Developers must declare and encode callback permissions, deploy core and periphery contracts, create a compatible hook address, initialize a pool, provide test assets and price feeds, replay trading activity, and calculate comparable outputs. Implementing these tasks separately for every policy undermines reproducibility and makes differences in surrounding infrastructure difficult to distinguish from differences in fee logic.

This paper addresses that systems gap with a configurable development and evaluation framework for Uniswap v4 dynamic-fee hooks. It connects a common Solidity contract

structure, local protocol deployment, historical-data replay, metric extraction, visualization, and project generation. The framework deliberately exposes the generated source: it assists development rather than replacing code review with an opaque configurator.

The contributions are:

- A modular contract and deployment architecture that separates fee-policy logic from Uniswap v4 lifecycle and bootstrap code.
- Configurable templates spanning block- and trade-adaptive behaviors, specifically implementing Exponential Moving Average (EMA) and Welford's online algorithm for variance-based fee calculation, alongside MEV-oriented and peg-stability policies.
- An integrated Foundry/Anvil experiment workflow that replays historical observations and collects pool-level outputs.
- A browser interface that presents policy parameters and source, exports a self-contained project, and visualizes simulation results.

The current artifact is evaluated as an engineering prototype. The available trace demonstrates that the full workflow executes, but lacks the fixed-fee baselines and repeated trials required to claim improved LP performance. This boundary is maintained throughout the paper.

II. Background and Related Work

A. AMM Fees and LP Exposure

For reserves x and y, a constant-product AMM maintains

$$xy = k, \tag{1}$$

apart from fees, rounding, and protocol accounting. If a trader supplies Δx and the proportional fee is f, the simplified output is

$$\Delta y = y - \frac{xy}{x + (1-f)\Delta x}. \tag{2}$$

Thus f affects both LP revenue and trader execution. Formal analyses show how constant-function markets track reference prices through arbitrage and generalize beyond two-asset products [2], [3], [16].

Let $V_{\mathrm{LP}}(T)$ be the terminal marked-to-market LP position, $V_{\mathrm{H}}(T)$ the value of holding the original assets, and $R_f(T)$ fee income in the same numeraire. A value-based loss and its fee-adjusted form are

$$IL(T) = V_{\mathrm{H}}(T) - V_{\mathrm{LP}}(T), \tag{3}$$

$$L_{\mathrm{net}}(T) = IL(T) - R_f(T). \tag{4}$$

This accounting distinction matters: an adaptive fee may compensate LPs through revenue without eliminating the stale-price mechanism that produces adverse selection. LVR makes that mechanism explicit by comparing the AMM with continuous rebalancing at the reference price [6], and recent work has further quantified the specific profitability zones where fee accumulation can turn impermanent loss into a sustainable gain [17].

A static pool sets $f_t = f$. A dynamic policy instead evaluates

$$f_t = \mathcal{F}(s_t, z_t; \theta), \tag{5}$$

where s_t is observable on-chain state, z_t optional external information, and θ a parameter vector. Directional policies return $f_t^{0\to1}$ and $f_t^{1\to0}$ separately, for example charging more when a swap moves the pool farther from an external reference.

B. Dynamic-Fee Research

Dynamic policies use different proxies for adverse conditions. Volatility-based mechanisms react to price variability; inventory policies account for reserve composition; oracle-informed policies use pool-to-reference deviation; and flow-based rules use recent volume, trade size, or direction. Lebedeva *et al.* compare block-level, transaction-level, and idealized oracle-informed mechanisms and find improved LP outcomes under their simulation assumptions [10]. Baggiani *et al.* identify two regimes in which fees balance arbitrage deterrence against attracting noise traders [11]. Campbell *et al.* similarly show that volatility and competing-venue costs jointly determine fee choice [12]. Recent agent-based analysis suggests that dynamic schedules may improve hedged LP profitability mainly by increasing compensation in toxic states, rather than mechanically eliminating LVR [13].

These works optimize or compare economic policies. Our contribution is complementary: a common implementation substrate for configuring, deploying, and replaying multiple fee rules as Uniswap v4 hooks. It enables a later controlled economic comparison while keeping the surrounding software path fixed.

The closest studies therefore occupy distinct levels. Formal CFMM work supplies the market foundation [2], [3]; LVR supplies an LP-risk benchmark [6]; and dynamic-fee studies supply candidate policies and economic hypotheses [10], [11], [13]. This paper operates at the systems level by making several rules deployable and observable through one protocol-specific path. Neither level replaces the other: a framework without controlled baselines cannot demonstrate economic benefit, while a fee model without reproducible implementation leaves integration effects unmeasured.

While protocols like Balancer V3, Curve V2, and Algebra Finance have introduced internal mechanisms for adaptive fees or volatility-based adjustments, these features are tightly coupled to their specific protocol architectures. They do not provide a unified, external management layer. Consequently, researchers and developers cannot easily deploy, configure, monitor, and compare multiple fee strategies across a standardized infrastructure. This framework fills that gap by providing a protocol-agnostic (within the Uniswap v4 ecosystem) external contour for strategy lifecycle management.

Table I condenses this distinction. Prior studies primarily contribute market models, LP-risk measures, or fee policies. The present work does not replace those results; it supplies the implementation and replay path needed to instantiate and compare policy families on Uniswap v4.

TABLE I
POSITIONING RELATIVE TO REPRESENTATIVE AMM AND DYNAMIC-FEE RESEARCH.

Work	Primary contribution	Relationship to this framework
Angeris *et al.* [2], [3]	Formal analysis and optimization of constant-function markets	Supplies the AMM foundation, not hook tooling
Milionis *et al.* [6]	LVR formulation for adverse-selection cost	Supplies an LP-risk benchmark for future evaluation
Lebedeva *et al.* [10]	Block-, deal-, and oracle-informed fee policies	Supplies a policy family represented by the templates
Baggiani *et al.* [11]	Stochastic-control formulation of dynamic fees	Supplies theoretical fee-design guidance
This work	Configurable contracts, deployment, replay, project export, and visualization	Supplies shared Uniswap v4 experimentation infrastructure

C. Uniswap v4 and External Data

Uniswap v4 stores pools in a singleton `PoolManager` and settles intermediate balance changes through flash accounting [14]. Hooks attach custom logic to initialization, liquidity, swap, and donation callbacks. Their enabled permissions are encoded in low-order address bits, so deployment must produce an address matching the declared callbacks [18], [19]. For dynamic pools, a hook can call `updateDynamicLPFee` or return a per-swap override from `beforeSwap` [15]. OpenZeppelin provides reusable bases for callback validation and fee management, including `BaseDynamicFee` and `BaseOverrideFee` [20].

Oracle-informed policies require a reference unavailable from the pool itself. Chainlink Data Feeds expose aggregated observations to contracts [21]. For two USD-denominated feeds, the external ratio is $r_t^{\text{ext}} = p_{0,t}/p_{1,t}$. Production logic must normalize decimals and reject invalid or stale rounds. In the framework, deterministic mocks reproduce historical observations; they validate integration logic, not the timing or fault behavior of a live oracle.

III. FRAMEWORK DESIGN

A. Layered Architecture

The framework has four layers. The *contract layer* contains hooks, base contracts, and interfaces; it computes directional fees, maintains policy state, validates callbacks, and interacts with `PoolManager`. The *deployment layer* uses `BaseScript`, `Deployers`, and `HookHelpers` to create mock tokens and feeds, deploy Uniswap artifacts and the selected hook, and initialize its pool. The *experiment layer* combines Forge tests, `Simulation.t.sol`, data utilities, and the result parser to replay observations and record price, volume, fees, holdings, and LP metrics. Finally, the *application layer* uses Flask and browser code to select and configure hooks, display source, export projects, run simulations, and present results. Foundry supplies Solidity-native compilation, tests, and scripts, while Anvil provides the local EVM [22], [23]; Flask implements the local HTTP interface [24].

The workflow is: (1) select a hook; (2) load its annotated description, parameters, and Solidity template; (3) set parameter values; (4) generate a configured copy; (5) export the project or choose a historical trace; (6) deploy the local infrastructure; (7) execute swaps; and (8) parse and visualize results. Policy code is therefore varied without rebuilding every supporting component.

The separation also defines explicit data boundaries. Configuration data flows from the browser to the generator; Solidity artifacts flow from the generator to the compiler and exporter; historical observations flow from the acquisition utility to the simulation; and emitted values flow from Forge to the parser and plots. The canonical hook sources are never simulation results, and the raw market observations are not inferred from the pool. This distinction prevents a configured constant, a reference-market input, and an endogenous pool output from being treated as interchangeable evidence.

Figure 1 makes the execution boundary explicit. A user choice and an annotated hook template enter the configurator, which produces a concrete Foundry project. The deployment scripts then instantiate the selected contract together with the local Uniswap dependencies. Historical observations enter only at the replay stage, where mock feeds and swap inputs drive contract execution. Finally, emitted values are parsed into metrics and figures. The generated project can also leave the workflow directly for independent inspection and execution; using the web interface is not required after export.

Two properties follow from this organization. First, policy comparison can reuse the same contract graph and replay machinery, reducing infrastructure variation between experiments. Second, every transition produces an inspectable artifact: configured Solidity, deployment logs, Forge output, or generated plots. This improves debuggability and makes it possible to identify whether a failure originates in generation, deployment, contract execution, data preparation, or result parsing.

B. Configuration and Generation

Figure 2 shows the configurator. For template C and parameters $\theta = [\theta_1, \ldots, \theta_n]$, generation produces

$$C_\theta = \mathcal{G}(C, \theta). \tag{6}$$

The current implementation substitutes exposed values into a generated copy and leaves the canonical template unchanged. The interface restricts hook selection to known templates, but textual substitution is not a typed Solidity transformation; this limitation motivates the typed schema discussed in Sec. V.

The code view updates with parameter changes (Fig. 3), allowing inspection before compilation. This transparency is

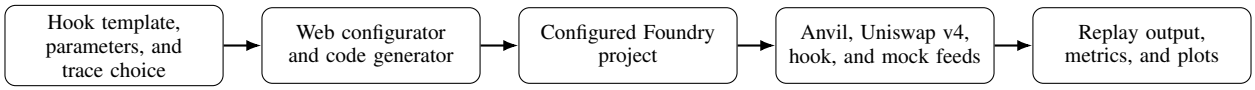

Fig. 1. End-to-end configuration, deployment, and replay workflow.

Fig. 2. Selecting a dynamic-fee template and its exposed parameters.

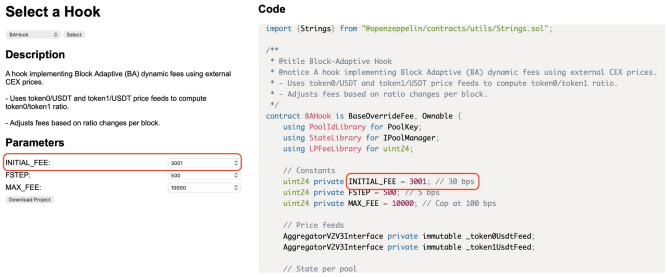

Fig. 3. Configured parameters reflected in the generated Solidity source.

important because fee units and bounds are protocol-specific: Uniswap v4 represents LP fees as `uint24` values in hundredths of a basis point [15]. Compilation and tests remain necessary; the interface is not a security verifier.

The generated artifact preserves the user-selected numeric values but does not automatically establish semantic validity. For example, a minimum fee can be syntactically valid while exceeding a configured maximum, and an oracle-dependent hook can compile with feeds for the wrong assets. The intended validation pipeline therefore has three levels: schema checks at configuration time, Solidity compilation and unit tests after generation, and deployment-time checks for network addresses, hook permissions, and pool configuration. The prototype implements the generation and test workflow; stronger typed constraints are identified as future work rather than assumed here.

C. Hook Interface and Lifecycle

The hook family reuses a fee-oriented base path consistent with OpenZeppelin's modular Uniswap hook design [20]. A representative block-adaptive implementation uses `poolManager()` to identify the authorized manager, `_getPriceRatio()` to normalize two feeds and obtain their ratio, `_afterInitialize()` to set initial state and directional fees, a fee selector for the swap direction, an owner-restricted setter where enabled, and `_afterSwap()` to advance state after eligible swaps. Shared callback checks prevent arbitrary callers from invoking lifecycle entry points directly.

Let r_b denote the external token-price ratio observed in block b:

$$\Delta_b = \frac{r_b - r_{b-1}}{r_{b-1}}. \tag{7}$$

The policy maps the change, prior directional fees, and parameters to

$$(f_b^{0\to1}, f_b^{1\to0}) = \mathcal{F}_{\mathrm{BA}}(\Delta_b, f_{b-1}^{0\to1}, f_{b-1}^{1\to0}; \theta), \tag{8}$$

subject to $f_{\min} \le f_b^d \le f_{\max}$. Updating at most once per block avoids reacting repeatedly to a shared reference observation; deal-adaptive policies can instead update after individual swaps.

The library exposes specific policy templates, including a block-adaptive hook (`BAHook`) that adjusts fees based on external price ratio changes, a deal-adaptive hook (`DAHook`) for transaction-level updates, and volatility-based hooks utilizing Exponential Moving Average (EMA) and Welford's online algorithm to react to trade volume variance.

Table II summarizes the policies at the abstraction level supported by the available implementation report. The MEV-oriented template should be understood as applying specialized fee logic associated with the corresponding scenario; the artifact does not demonstrate complete detection or prevention of MEV. Likewise, the peg-stability template supplies peg-oriented behavior but is not evidence that a peg can be maintained under arbitrary liquidity or market shocks.

The common per-swap path can be summarized as follows. First, the callback verifies the manager and resolves the swap direction. Second, the hook reads the policy state and, when required, normalized external observations. Third, it evaluates the policy and bounds the result. Fourth, it returns or stores the protocol-formatted fee. Finally, the post-swap callback updates only the state permitted by the policy's granularity. Separating selection from state advancement makes repeated execution within a block explicit and testable.

D. Deployment and Export

Hook permissions are derived from the deployed address. The deployment path must therefore mine a compatible `CREATE2` salt, or use Foundry test facilities to place code at a compatible address locally [19]. The framework then deploys or resolves `PoolManager`, position management, and routing artifacts; creates mock tokens and price feeds; deploys the chosen hook; constructs the pool key; initializes price; and adds test liquidity. The expected permission bitmap is checked because a wrong salt can disable required callbacks or enlarge the hook's attack surface [25].

The exported archive contains the configured hook, interfaces, scripts, tests, dependency declarations, remappings,

TABLE II
DYNAMIC-FEE TEMPLATES EXPOSED BY THE FRAMEWORK.

Template	Update granularity	Primary signal or condition	Framework role
`BAHook`	At most once per block	Change in an external token-price ratio	Demonstrates block-adaptive directional fees
`DAHook`	Individual trade	Deal-level state and swap characteristics	Demonstrates transaction-level adaptation
`ABHook`	Block-level	External ratio with asymmetric response	Demonstrates distinct fees by swap direction
MEV-charge hook	Policy-specific	Conditions encoded by the MEV-oriented rule	Provides a specialized charging template and tests
Peg-stability hook	Policy-specific	Deviation from a target or peg-related state	Provides a template for peg-oriented fee behavior

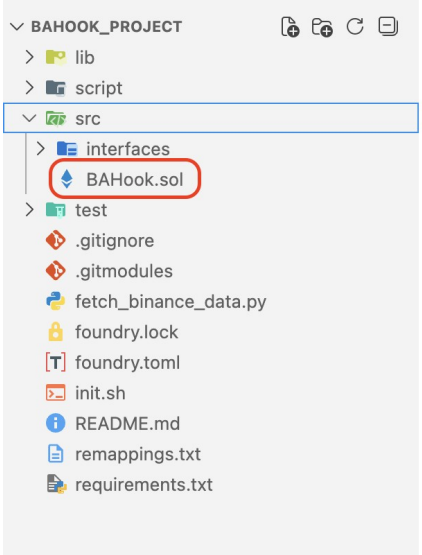

Fig. 4. Generated Foundry project for a configured hook.

historical-data utility, and instructions (Fig. 4). This is more reproducible than exporting a single contract, although complete experiment replication also requires pinned dependency revisions, compiler version, input data, initial pool state, and parameters.

For a replay to be independently identifiable, its manifest should include

$$E = \langle C_\theta, D, P_0, L_0, W, S \rangle, \tag{9}$$

where C_θ is the configured contract, D the dependency and compiler revisions, P_0 initial pool price, L_0 initial liquidity, W the market-data window, and S the swap-generation configuration. The current archive contains the executable project structure; recording all elements of E is a concrete extension needed for artifact-level reproducibility.

To ensure strict reproducibility and auditability, the framework's off-chain layer utilizes a relational database (e.g., `PostgreSQL`) to maintain a complete history of parameter changes (`HookParamsHistory`), indexed blockchain events (`ChainEvent`), and pool state snapshots (`PoolStateSnapshot`). This allows any historical replay to be deterministically linked to the exact configuration and market state at the time of execution.

E. Testing Strategy

Testing is organized across three layers to ensure robustness. First, smart-contract tests (via Foundry) verify fee direction, bounds, update granularity, and edge values for each generated hook. Second, backend integration tests (in Go) validate the API layer, database state transitions, and historical data ingestion. Finally, end-to-end (e2e) tests (e.g., via Playwright) verify the complete user workflow, from web-based parameter configuration to project export and simulation execution. This multi-layer approach ensures that failures can be precisely localized to generation, deployment, contract execution, or data parsing.

The report identifies both common hook tests and specialized suites such as `MEVChargeHookFees.t.sol` and `PegStabilityHook.t.sol`. The simulation suite is distinct from unit tests. Unit tests cover fee direction, bounds, update granularity, and edge values. Deployment tests verify addresses, permissions, dependencies, and the pool key. Integration tests initialize liquidity and exercise callbacks through successful swaps, while replay tests validate observation ingestion, repeated execution, metrics, and plots. Adversarial cases should reject an unauthorized caller, stale feed, invalid parameter, and extreme input. Passing a replay is not a substitute for unit coverage because a long scenario may never reach these boundaries.

For a block-adaptive hook, a minimal transition suite should include $\Delta_b = 0$, positive and negative changes, values that would cross both fee bounds, two swap directions, and multiple swaps assigned to the same block. It should verify that a same-block second swap does not advance block-level state unexpectedly. A deal-adaptive hook instead requires consecutive swaps to demonstrate that transaction-level updates occur. Oracle-informed tests should vary feed decimals, timestamps, sign, and the relative ordering of the two token prices. Owner-controlled setters need both authorized and unauthorized callers.

Generated projects also require differential checks against their templates. For a parameter vector θ, the generated contract should differ only at declared substitution locations, compile under the recorded compiler, and retain the same callback permission set. A simple text comparison can identify accidental edits, while runtime tests confirm that the new constants affect the intended state. These checks are especially

important because template generation occurs before deployment, after which an error may be immutable.

Finally, deterministic replay requires control over the chain and data context. Anvil fixes the local execution environment; the replay should additionally record block timestamps, observation order, token decimals, initial balances, liquidity, and any generated trade sizes. Without these fields, two executions can use the same nominal month and pair but produce different results. The manifest in Eq. (9) provides the core identifier, while an input checksum and event-level output would make exact comparison practical.

IV. Replay and Prototype Evaluation

A. Replay Procedure and Metrics

The experiment form accepts a hook, year, month, and token pair. The backend obtains Binance kline observations [26], transforms them into the input consumed by `Simulation.t.sol`, and launches Forge. The test contract initializes mock tokens and a pool, acts as trader and LP, updates price-feed mocks, executes the swap sequence, and emits parseable output. The application extracts swap count, volume, directional fees, fee income, holdings, LP value, impermanent loss, and net loss, then generates the market and pool plots.

For swaps $i = 1, \ldots, N$, let q_i be the simulator's volume measure and $a_{i,j}^{f}$ the fee amount collected in token j. The direct aggregates are

$$Q = \sum_{i=1}^{N} q_i, \qquad F_j = \sum_{i=1}^{N} a_{i,j}^{f}. \tag{10}$$

Token-denominated fees should remain separate unless a valuation price and time are declared. Once all values use a common numeraire, Eqs. (3)–(4) provide the gross and fee-adjusted LP measures. The further conditions a conclusive comparison would require are set out in Sec. V, and none is inferred from the prototype.

Historical CEX observations drive the offline scenario; Chainlink-compatible mocks provide the contract-readable interface. They are intentionally distinct roles. Replaying Binance data through a mock feed does not reproduce a live feed's source aggregation, heartbeat, deviation threshold, or latency.

B. Prototype Execution

The supplied execution contains 1,492 simulated swaps. Because the report does not identify a volume numeraire, no USD interpretation is assigned to the aggregate volume or token-denominated fee income.

To demonstrate that the framework correctly links market conditions to fee adjustments, Figure 5 visualizes the core mechanism of the dynamic-fee hook in action over a representative window of 1,492 simulated swaps. The top panel shows the SHIB/ETH price ratio serving as the oracle input. The bottom panel shows the resulting dynamic fee parameter (in basis points). This two-panel view explicitly demonstrates the causal chain: as the price ratio exhibits higher variance, the hook scales the fee upward from the base rate, directly reflecting the calculated market volatility while respecting the configured bounds. This confirms that the integration path functions as designed, even though this single trace does not constitute a full economic baseline comparison.

The execution verifies the integration path: configure a hook, deploy local contracts, inject observations, execute swaps, collect contract output, and render results. It does not establish economic superiority. The artifact provides no same-trace fixed-fee baseline, repeated trials, uncertainty intervals, gas measurements, or complete specification of trade sizing and data cleaning. Accordingly, the paper claims feasibility rather than reduced LP loss.

V. Security, Validity, and Discussion

A. Security Considerations

Dynamic-fee hooks execute inside the swap lifecycle and enlarge the trusted code surface. Production implementations should restrict setters, accept callbacks only from the designated `PoolManager`, enforce protocol and policy fee bounds, normalize oracle decimals, reject stale or nonpositive rounds, and handle division and casts safely. Address-permission validation is essential. Uniswap's security guidance emphasizes that incorrectly encoded permissions or custom deltas can produce unintended execution paths [25].

The web layer crosses a separate trust boundary. Hook names should be allowlisted; parameter types and ranges should be validated before substitution; generated files should be written to isolated temporary locations; and simulator arguments must not become shell fragments. Resource limits are needed because compilation and replay are user-triggered operations. Exported projects should pin revisions to prevent later dependency changes from silently altering a result.

Between the contract and application layers sits a data boundary: the implementation must check feed identity, sign, decimals, and timestamp, and define behavior for unavailable observations. Contract- and data-boundary threats admit code-level tests; application-layer risks and historical-replay bias instead require common baselines, multiple regimes, and explicit experimental reporting.

B. Validity and Design Trade-offs

Local execution provides deterministic debugging but not public-chain market microstructure. Historical replay omits mempool visibility, block-builder ordering, latency, gas competition, strategic arbitrage, and endogenous routing. Results also depend on initial price and liquidity, token decimals, observation frequency, trade sizes, valuation time, and the share of informed flow. One pair and interval cannot establish generality, particularly across volatile, stablecoin, and shallow-liquidity markets.

The framework's main benefit is separation of concerns: researchers can change $\mathcal{F}$ in Eq. (5) while retaining deployment, ingestion, and analysis. Generated source remains inspectable, and mocks make integration tests repeatable. The trade-off is

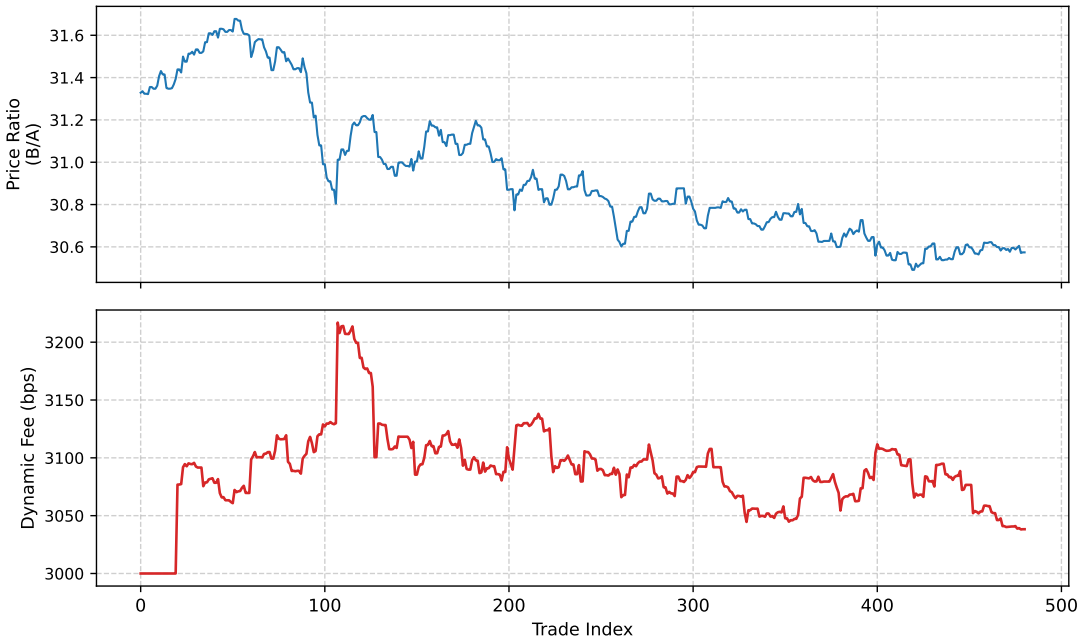

Fig. 5. Core mechanism of the dynamic-fee hook in action. **(Top)** Price ratio (SHIB/ETH) showing market movement. **(Bottom)** The resulting dynamic fee parameter (in basis points), demonstrating how the policy scales the fee in response to underlying market volatility.

that textual template substitution is less safe and expressive than a typed policy language; mock feeds validate calls but not production oracle behavior; and console parsing is less robust than a structured event-level result format.

Developing robust backtesting infrastructure for AMM strategies is an active area of research [27]. A controlled economic study within such a framework should hold the trace, initial reserves, liquidity range, valuation convention, and random seed constant while varying only the policy. It should compare all hooks with static-fee baselines and report LP value, fee income, IL or LVR, net outcome, retained volume, mean and distribution of fees, gas overhead, and runtime. Multiple pairs, volatility regimes, and repeated windows are required for uncertainty estimates. These additions are future evaluation work, not evidence inferred from the current trace.

The next implementation steps are therefore a typed parameter schema, containerized and version-pinned experiments, event-level result export, property-based tests for fee bounds and callback authorization, stale-oracle fault injection, and testnet validation. Together they would turn the current development framework into a stronger comparative research testbed. Furthermore, the framework is designed to easily accommodate emerging fee paradigms, such as trade-size-aware Impermanent-Loss Trimming (ILT) mechanisms and path-independent fee structures that preserve composability guarantees under transaction fragmentation [28].

VI. Conclusion

This paper presented an end-to-end framework for configuring, deploying, and replay-testing dynamic-fee hooks on Uniswap v4. Shared contracts and scripts isolate policy logic from protocol bootstrap; the web interface exposes parameters and source; and the experiment path connects historical observations to local execution and metrics. A 1,492-swap ETH/SHIB trace demonstrates that the integrated workflow operates. It does not prove that a dynamic policy outperforms static fees. The contribution is transparent, reusable infrastructure that enables such policies to be implemented and, with controlled baselines and broader traces, compared reproducibly.

References

[1] Sam M. Werner, Daniel Perez, Lewis Gudgeon, Ariah Klages-Mundt, Dominik Harz, and William J. Knottenbelt. SoK: Decentralized finance (DeFi). In *Proc. 4th ACM Conf. Advances in Financial Technologies*, pages 30–46, 2022.

[2] Guillermo Angeris, Hsien-Tang Kao, Rei Chiang, Charlie Noyes, and Tarun Chitra. An analysis of uniswap markets, 2019.

[3] Guillermo Angeris, Akshay Agrawal, Alex Evans, Tarun Chitra, and Stephen Boyd. Constant function market makers: Multi-asset trades via convex optimization. In *Handbook on Blockchain*, pages 415–444. Springer, 2022.

[4] Hayden Adams, Noah Zinsmeister, and Dan Robinson. Uniswap v2 core. White paper, 2020.

[5] Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, and Dan Robinson. Uniswap v3 core. White paper, 2021.

[6] Jason Milionis, Ciamac C. Moallemi, Tim Roughgarden, and Anthony Lee Zhang. Automated market making and loss-versus-rebalancing, 2022.

[7] Zhou Fan, Francisco Marmolejo-Cossio, Daniel J. Moroz, Michael Neuder, Rithvik Rao, and David C. Parkes. Strategic liquidity provision in uniswap v3, 2021.

[8] Roman Vlasov, Vladimir Gorgadze, and Artem Barger. The Impact of the Exchange Fees on Impermanent Loss of Liquidity Providers for Conservative Automated Market Makers. *The Journal of The British Blockchain Association*, may 27 2025.

[9] Robin Fritsch. A note on optimal fees for constant function market makers. In *Proceedings of the 2021 ACM CCS Workshop on Decentralized Finance and Security*, CCS '21, page 9–14. ACM, November 2021.

[10] Irina Lebedeva, Dmitrii Umnov, Yury Yanovich, Ignat Melnikov, and George Ovchinnikov. Dynamic fee for reducing impermanent loss in decentralized exchanges, 2025.
[11] Leonardo Baggiani, Martin Herdegen, and Leandro Sánchez-Betancourt. Optimal dynamic fees in automated market makers, 2025.
[12] Steven Campbell, Philippe Bergault, Jason Milionis, and Marcel Nutz. Optimal fees for liquidity provision in automated market makers, 2025.
[13] Daniele Maria Di Nosse and Fabrizio Lillo. Mitigating adverse selection in concentrated liquidity AMMs with dynamic fees: An agent-based model approach, 2026.
[14] Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, and Dan Robinson. Uniswap v4 core. White paper, 2024.
[15] Uniswap Labs. Dynamic fees. Uniswap v4 Developer Documentation, 2026. Accessed: 2026-07-17.
[16] Guillermo Angeris and Tarun Chitra. Improved price oracles: Constant function market makers. In *Proc. 2nd ACM Conf. Advances in Financial Technologies*, pages 80–91, 2020.
[17] Ignat Melnikov, Roman Vlasov, Vladimir Gorgadze, Andrey Seoev, and Yury Yanovich. From impermanent loss to sustainable gain: Quantifying profitability zones for liquidity providers on dex. In *IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, 2026. to appear.
[18] Uniswap Labs. Hooks. Uniswap v4 Developer Documentation, 2026. Accessed: 2026-07-17.
[19] Uniswap Labs. Hook deployment. Uniswap v4 Developer Documentation, 2026. Accessed: 2026-07-17.
[20] OpenZeppelin. Uniswap hooks. Developer documentation, 2026. Accessed: 2026-07-17.
[21] Chainlink Labs. Chainlink data feeds. Developer documentation, 2026. Accessed: 2026-07-17.
[22] Foundry Contributors. Foundry documentation: Forge. Developer documentation, 2026. Accessed: 2026-07-17.
[23] Foundry Contributors. Foundry documentation: Anvil. Developer documentation, 2026. Accessed: 2026-07-17.
[24] Pallets Projects. Flask documentation. Developer documentation, 2026. Accessed: 2026-07-17.
[25] Uniswap Labs. Uniswap v4 security framework. Developer documentation, 2026. Accessed: 2026-07-17.
[26] Binance. Spot API: Kline/candlestick data. Developer documentation, 2026. Accessed: 2026-07-17.
[27] Andrey Urusov, Rostislav Berezovskiy, and Yury Yanovich. Backtesting framework for concentrated liquidity market makers on uniswap v3 decentralized exchange. *Blockchain: Research and Applications*, 6(1):100256, 2025.
[28] Andrey Voronin, Roman Vlasov, Vladimir Gorgadze, Andrey Seoev, and Yury Yanovich. Characterizing path-independent fees: A route to zero impermanent loss in cpmms. In *2026 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pages 1–7, 2026.